

Stevens Institute of Technology

Master Of Science in Computer Science

CS590 Algorithm

Name : Goti Parth Rameshbhai

CWID : 20005769

Hw2 : Assignment_Report_2

Part 1

Recurrences:

1) $T(n) = T(n-3) + 3 \lg n$. Our guess: $T(n) = O(n \lg n)$.

Show $T(n) \leq cn \lg n$ for some constant $c > 0$.

$\lg n$ is monotonically increasing for $n > 0$

Answer:

For $T(n) = T(n-3) + 3 \lg n$

$$T(n) \leq cn \lg n$$

Our Guess:

$$T(n) = O(n \lg n)$$

$$T(n) \leq cn \lg n$$

For each $c \geq 1$ and $n^0 < n$

Prove:

$$n = 1 \Rightarrow n \log n = 1 \cdot 0 = 0$$

In Step:

$$T(n-3) = (n-3) \lg (n-3)$$

$$T(n) = T(n-3) + 3 \lg n$$

$$\leq c(n-3) \lg (n-3) + 3 \lg n$$

$$\leq cn \lg (n-3) - 3c \lg (n-3) + 3 \lg n$$

Here $\lg n$ is monotonically increasing for $n > 0$

So, $\lg n$ is greater than $\lg (n-3)$

$$\leq cn \lg n - 3c \lg n + 3 \lg n$$

As we Consider,

$$c \geq 1$$

Remove terms of lower order,

$$\leq cn \lg n$$

So, $T(n) \leq cn \lg n$ proves our guess.

2) $T(n) = 4T(n/3) + n$. Our guess: $T(n) = O(n \log_3 4)$

Show $T(n) = cn \log_3 4$ for some constant $c > 0$.

Answer:

For $T(n) = 4T(n/3) + n$

$T(n) = cn \log_3 4$

Our Guess:

$T(n) = O(n \log_3 4)$

$T(n) \leq cn \log_3 4$

For each $c \Rightarrow 1$ and $n^0 < n$

Prove:

$n = 1 \Rightarrow 1 \log_3 4 \Rightarrow 1$

In Step:

$T(n/3) = (n/3) \log_3 4$

$T(n) = 4T(n/3) + n$

$\leq 4c(n/3) \log_3 4 + n$

$\leq 4c4(\log_3(n/3)) + n$

$\leq 4c4(\log_3(n)) - (\log_3(3)1) + n$

$\leq 4c4(\log_3(n)) - 1 + n$

$\leq 4c(4(\log_3(n))) / 4 + n$

$\leq c4(\log_3(n)) + n$

$\leq cn \log_3 4 + n$

After Subtracting term of lower order,

Our guess fails That's why,

Taking another new guess,

$T(n) \leq cn \log_3 4 - dn$ Proving,

$T(n) = 4T(n/3) + n$

$T(n) \leq 4c(n/3) \log_3 4 - ((4/3)dn) + n$

$\leq 4c4(\log_3(n)) - (\log_3(3)1) + n - (4/3)dn$

$\leq 4c4(\log_3(3^n) - 1) + n - (4/3)dn$

$\leq 4c(4(\log_3(3^n))) / 4 + n - (4/3)dn$

$$\leq c_4 (\log 3^n) + n - (4/3)dn$$

$$\leq cn (\log 3/4) + n - (4/3)dn$$

As we Consider,

$$(-dn) / 3 + n \leq 0,$$

We will get, $d \geq 3$.

After Subtracting term of lower order,

$$T(n) \leq cn (\log 3/4) - 3n$$

So, $T(n) \leq cn (\log 3/4)$ proves our 2nd guess.

$$3) T(n) = T(n/2) + T(n/4) + T(n/8) + n.$$

Our guess: $T(n): O(n)$.

Show $T(n) \leq cn$ for some constant $c > 0$.

Answer:

$$\text{For } T(n) = T(n/2) + T(n/4) + T(n/8) + n,$$

$$T(n) \leq cn$$

Our Guess:

$$T(n) = O(n)$$

$$T(n) \leq cn$$

Prove:

$$n = 1 \Rightarrow 1$$

In Step:

$$T(n/2) = (n/2)$$

$$T(n/4) = (n/4)$$

$$T(n/8) = (n/8)$$

$$T(n) = T(n/2) + T(n/4) + T(n/8) + n$$

$$\leq c(n/2) + c(n/4) + c(n/8) + n$$

$$\leq (4cn + 2cn + cn + 8n) / 8$$

$$\leq ((7c + 8)n) / 8$$

As we Consider,

$$(7c + 8) / 8 \geq 1$$

$$c \geq T(n) \leq c'n$$

So, $T(n) \leq cn$, proves our guess.

4) $T(n) = 4T(n/2) + (n^2)$. Our Guess: $T(n) = O(n)$.

Show $T(n) \leq c(n^2)$ for some constant $c > 0$.

Answer:

$$\text{For } T(n) = 4T(n/2) + (n^2)$$

Our Guess:

$$T(n) = O(n)$$

$$T(n) \leq c(n^2)$$

Prove:

$$T(1) = 1$$

In Step:

$$T(n/2) = n/2$$

$$T(n) = 4T(n/2) + (n^2)$$

$$\leq 4c((n/2)^2) + (n^2)$$

$$\leq 4c(n^2)/4 + (n^2)$$

$$\leq c(n^2) + (n^2)$$

After Subtracting term of lower order,

Our guess fails That's why,

Taking another new guess,

Using Recurrence Tree to evaluate our guess,

We get,

$$T(n) \leq c_1(n^2) (\log n - ((c_2) n))$$

Solve,

$$T(n) = 4T(n/2) + (n^2)$$

$$T(n) \leq (4(c_1)((n^2) (\log n/2)) / 4 - (4(c_2) n) / 4 + (n^2)$$

$$\leq (c_1((n^2) (\log n/2))) - ((c_2) n) + n^2$$

$$\leq (c_1((n^2) \log n)) - (c_1((n^2) \log 2)) - ((c_2) (n)) + (n^2)$$

$$\leq (c_1((n^2) \log n)) - ((c_1 \log 2 - 1) (n^2)) - ((c_2) (n))$$

$T(n)$ = Consider,

$$c_2 \geq 0 \text{ and } c_1 \log 2 - 1 \geq 0,$$

$$c_1 \geq 1 / \log 2$$

So, $T(n) \leq (c_1 (n^2)) (\log (n - ((c_2) (n))))$, proves our 2nd guess.

Part 2: Radix-sort on strings

1) Implement an insertion sort algorithm for strings that sorts a given array of strings according to the character at position d . It is necessary to include the length of each string (*array A len*) as it is unclear whether or not the digit d exists. A non-existing digit d is interpreted as a 0 in the sorting process. Add a parameter *int d* and *int* A_len* to the algorithm arguments and modify the given insertion sort algorithm accordingly. This algorithm should be implemented in the method below:

```
void insertion_sort_digit(char** A, int* A_len, int l, int r, int d)
```

Answer:

I have developed the function called `insertion_sort_digit` which takes in arguments such as *A* as 2-dimension vector array which stores the *n* number of strings of various length from 0 to Max. I have also created function called `string_compare_new(char* s1, char* s2, int d, int length)` which compares 2 string at a individual digit and returns the comparison. And from the output of the new created function, I had swapped values.

2) Use this modified insertion sort algorithm to implement radix sort for an array of strings. Measure the runtime performance for arrays of random strings 5 times for every combination of array size $n = 2500; 5000; 7500; 10000$ and length of the random strings $m = 25; 35; 45$. Discuss your results. (You might have to adjust the value for n dependent on your computers speed, but allow each test to take up to a couple of minutes). This algorithm should be implemented in the method below:

```
void radix_sort_is(char** A, int* A_len, int n, int m)
```

Answer:

I had only run the `insertion_sort_digit` function from last digit to its previous one and so on. And after running that function for last time with first digit, we get output which is the sorted array of strings. But, the time required is higher enough because, Time Complexity is $O((n^2)(d))$ here d is number of digits and n is input.

3) Develop a counting sort algorithm for strings that sorts a given array of strings according to the character at position d . As for the insertion sort on digit d , a non-existing digit d is treated as a 0 throughout the counting sort. This algorithm should be implemented in the method below:

```
void counting_sort_digit(char** A, int* A_len, char** B, int* B_len, int n, int d)
```

Answer:

The counting sort is linear time sorting algorithm which sorts using the count array where it contains counts of each character and each count is added to previous one to get the index position of string in new created output of array.

4) Use this new counting sort algorithm to implement radix sort for an array of strings. Measure the runtime performance for arrays of random strings 5 times for every combination of array size $n = 50000; 100000; 500000; 750000$ and length of the random strings $m = 25; 35; 45$. Discuss your results. (You might have to adjust the value for n dependent on your computers speed, but allow each test to take up to a couple of minutes). This algorithm should be implemented in the method below:

```
void radix_sort_cs(char** A, int* A_len, int n, int m)
```

Answer:

Implemented counting sort for each and every digit starting from last to the first. The time complexity is linear time because of stable counting sort which also takes linear time to operate. The total time complexity is $O((n+256) \cdot d)$ where n is input size and d is digit of input size. Which is to be $O(n)$ consider 256 and d as constant and would not affect for larger input values.

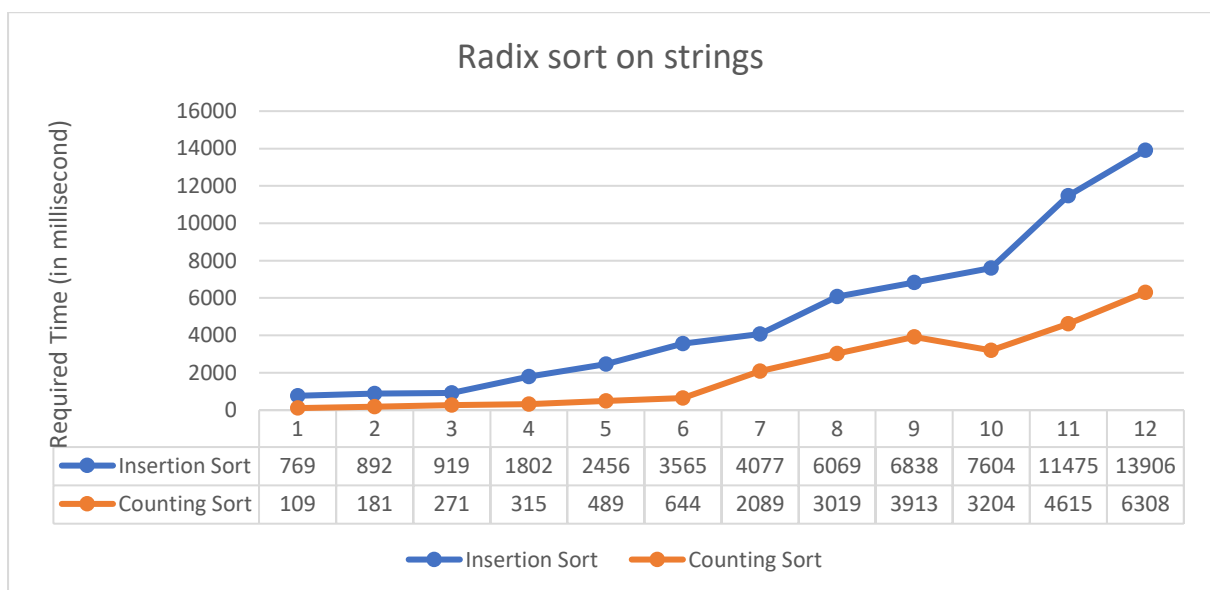
Radix Sort with Insertion Sort:

Random Array Length	String Length	Time (in millisecond)
2500	25	769
	35	892
	45	919
5000	25	1802
	35	2456
	45	3565
7500	25	4077
	35	6069
	45	6838
10000	25	7604
	35	11475
	45	13906

Radix Sort with Counting Sort:

Random Array Length	String Length	Time (in millisecond)
50000	25	109
	35	181
	45	271
100000	25	315
	35	489
	45	644
500000	25	2089
	35	3019
	45	3913
750000	25	3204
	35	4615
	45	6308

Comparison chart of both radix sort using Insertion sort and Counting Sort:



Here, we can see that for larger input values, difference is very faster for counting sort. But, for smaller values, the counting sort is not very faster. We can see for larger input values, the difference between them rises exponentially in terms of time complexity.